



G L O B A L R A I N

Practices for Secure Software Report

Table of Contents

PRACTICES FOR SECURE SOFTWARE REPORT.....	1
DOCUMENT REVISION HISTORY	3
CLIENT.....	3
DEVELOPER.....	4
1. ALGORITHM CIPHER	4
2. CERTIFICATE GENERATION	4
3. DEPLOY CIPHER	4
4. SECURE COMMUNICATIONS	5
5. SECONDARY TESTING	5
6. FUNCTIONAL TESTING	6
7. SUMMARY	7
8. INDUSTRY STANDARD BEST PRACTICES	7

Document Revision History

Version	Date	Author	Comments
1.0	Oct. 14, 2023	Jarrale Butts	

Client



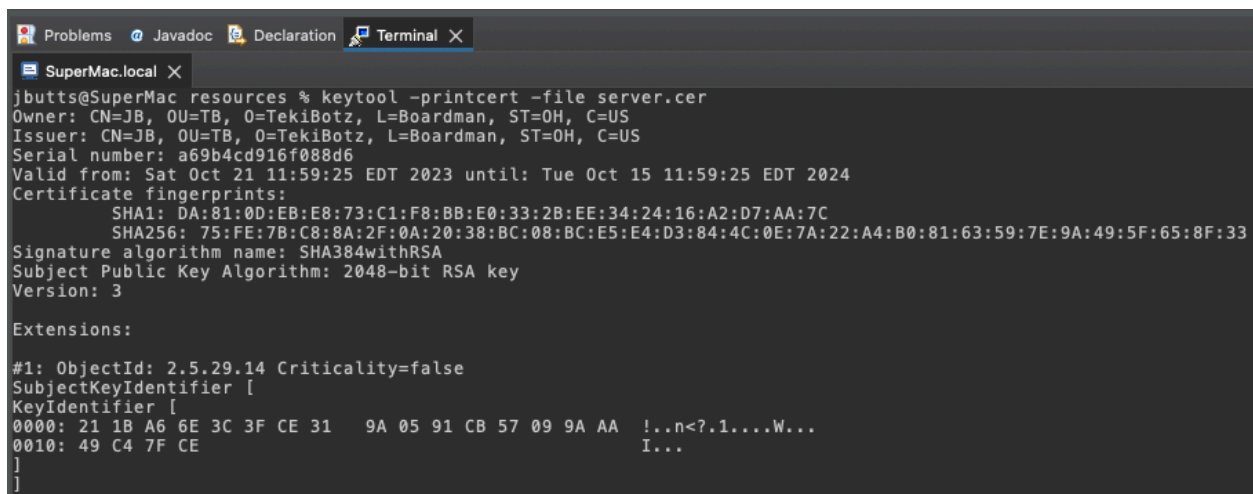
Developer

Jarrale Butts

1. Algorithm Cipher

My recommendation for an appropriate encryption algorithm cipher for Artemis Financial to add a file verification step to their web application to ensure secure communications is the Secure Hash Algorithm (SHA-256). SHA-256 is a cryptographic hash function (mathematical function that converts a numerical input value into another compressed numerical value) designed to withstand collision attacks which takes an input and produces a hash value/Checksum of 256 bits, usually represented as a hexadecimal number. Random numbers are used for things such as creating the keys for the hash function used to decrypt encrypted data. Symmetric keys (secret keys) are generally used for encryption, where the same key is used to encrypt and decrypt data. Asymmetric keys (public keys) are generally used for signing data where one key is kept secret while the other is made public to anyone to validate a signature. Currently, we have many safe and strong encryption algorithms and protocols to keep data safe in transit and at rest.

2. Certificate Generation



```
Problems Javadoc Declaration Terminal X
SuperMac.local X
jbutts@SuperMac resources % keytool -printcert -file server.cer
Owner: CN=JB, OU=TB, O=TekiBotz, L=Boardman, ST=OH, C=US
Issuer: CN=JB, OU=TB, O=TekiBotz, L=Boardman, ST=OH, C=US
Serial number: a69b4cd916f088d6
Valid from: Sat Oct 21 11:59:25 EDT 2023 until: Tue Oct 15 11:59:25 EDT 2024
Certificate fingerprints:
    SHA1: DA:81:0D:EB:E8:73:C1:F8:BB:E0:33:2B:EE:34:24:16:A2:D7:AA:7C
    SHA256: 75:FE:7B:C8:8A:2F:0A:20:38:BC:08:BC:E5:E4:D3:84:4C:0E:7A:22:A4:B0:81:63:59:7E:9A:49:5F:65:8F:33
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:
#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 21 1B A6 6E 3C 3F CE 31    9A 05 91 CB 57 09 9A AA    !..n<?.1....W...
0010: 49 C4 7F CE                    I...
]
]
```

3. Deploy Cipher

```

1 package com.snhu.sslserver;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.web.bind.annotation.RequestMapping;
6 import org.springframework.web.bind.annotation.RestController;
7 import java.security.MessageDigest;
8
9 @SpringBootApplication
10 public class SslServerApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(SslServerApplication.class, args);
14     }
15 }
16
17 //FIXME: Add route to enable check sum return of static data example: String data = "Hello World Check Sum!";
18
19 @RestController
20 class ServerController{
21     @RequestMapping("/hash")
22     public String myHash() throws Exception {
23         String name = "Jarrale Butts!";
24
25         // Created an object of the MessageDigest class using the SHA-256 algorithm
26         MessageDigest md = MessageDigest.getInstance("SHA-256");
27         // Initialize the object with the data string.
28         md.update(name.getBytes());
29         // Generate the hash value.
30         byte[] hash = md.digest();
31         // Convert the hash value to hex.
32         String hexToHash = bytesToHex(hash);
33
34         return "<p>data:" + name + "<br>This message will display my SHA-256: CheckSum Value:" + hexToHash;
35     }
36
37     private static String bytesToHex(byte[] hash) {
38         StringBuilder hexString = new StringBuilder();
39         for (byte b : hash) {
40             hexString.append(String.format("%02x", b));
41         }
42         return hexString.toString();
43     }
44 }

```

4. Secure Communications



data:Jarrale Butts!
This message will display my SHA-256: CheckSum Value:ed5cda6c42e08e74a3144f0d04354eeb00f1a23a2176cc1d2f4d81d49b7876a8

5. Secondary Testing

```

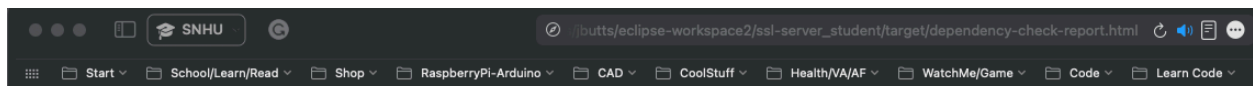
1 package com.snhu.sslserver;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.web.bind.annotation.RequestMapping;
6 import org.springframework.web.bind.annotation.RestController;
7 import java.security.MessageDigest;
8
9 @SpringBootApplication
10 public class SslServerApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(SslServerApplication.class, args);
14     }
15 }
16
17
18 @RestController
19 class ServerController {
20     @RequestMapping("/hash")
21     public String myHash() throws Exception {
22         String name = "Jarrale Butts!";
23
24         // Created an object of the MessageDigest class using the SHA-256 algorithm
25         MessageDigest md = MessageDigest.getInstance("SHA-256");
26         // Initialize the object with the data string.
27         md.update(name.getBytes());
28         // Generate the hash value.
29         byte[] hash = md.digest();
30         // Convert the hash value to hex.
31         String hexToHash = bytesToHex(hash);
32
33         return "<p>data: " + name + "<br>This message will display my SHA-256: CheckSum Value: " + hexToHash;
34     }
35
36     private static String bytesToHex(byte[] hash) {
37         StringBuilder hexString = new StringBuilder();
38         for (byte b : hash) {
39             hexString.append(String.format("%02x", b));
40         }
41         return hexString.toString();
42     }
43 }

```

```

[INFO] Installing /Users/jbutts/eclipse-workspace2/ssl-server_student/target/ssl-server-0.0.1-SNAPSHOT.jar to /Users/jbutts/.m2/repository/com/snhu/ssl-server/0.0.1-SNAPSHOT/ssl-server-0.0.1-SNAPSHOT.jar
[INFO] Installing /Users/jbutts/eclipse-workspace2/ssl-server_student/pom.xml to /Users/jbutts/.m2/repository/com/snhu/ssl-server/0.0.1-SNAPSHOT/ssl-server-0.0.1-SNAPSHOT.pom
[INFO] BUILD SUCCESS
[INFO] Total time: 04:06 min
[INFO] Finished at: 2023-10-21T15:19:58-04:00
[INFO]

```





DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, as reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) |
 [Suppressing false positives](#) |
 [Getting Help: github issues](#)

[Sponsor](#)

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information ([show less](#)):

- dependency-check version: 8.4.0
- Report Generated On: Sat, 21 Oct 2023 15:19:57 -0400
- Dependencies Scanned: 49 (31 unique)
- Vulnerable Dependencies: 15
- Vulnerabilities Found: 112
- Vulnerabilities Suppressed: 0
- NVD CVE Checked: 2023-10-21T15:19:46
- NVD CVE Modified: 2023-10-21T14:00:01
- VersionCheckOn: 2023-10-10T21:21:59
- kev.checked: 1697915992

6. Functional Testing

I found my code to be free of syntactical, logical, and security vulnerabilities, therefore I didn't make any changes in refactoring my code.

